

Faculty of Engineering
Ain Shams University



AI-Powered Predictive Cloud Operations and Autonomous Kubernetes Scaling

Senior Project Proposal

Authors:	Senior Project Team
Supervisor:	Dr. Wagdy Refaat Anis
Submission Date:	September 2026
Academic Year:	2026/2027

Project Team

Computer and Artificial Intelligence Engineering Program

Student name	Student ID
Menna Osama Mohamed	2300515
Ahmed Omar Ahmed	2300697
Samy Ahmed Mahmoud Samy	24P0194
Youssef Mohamed Abdelrazek	2300988
Mostafa Ahmed Mohamed	2300389
Ahmed Mohamed Elsayed	23P0035
Jana Mohamed Omar	2301032

Supervisor: Dr. Wagdy Refaat Anis

Study level: Fourth Year (Senior Year)

Abstract

This proposal combines two graduation project topics into one end-to-end cloud operations platform.

The first phase implements Project 9: Kubernetes Autoscaling with Machine Learning Based on Traffic Load Prediction. The second phase implements Project 1: AI-Generated Response and Decision-Making Solution for Cloud Operations.

Phase 1 begins with an existing online dataset. The dataset is already available and will not be extracted from Kubernetes. The team will clean and analyze the data, perform exploratory data analysis, engineer time-series features, and train a machine learning model to predict future traffic load. In parallel with model development, the team will prepare a Kubernetes environment, deploy a test application, configure monitoring, and implement a reactive autoscaling baseline. The selected prediction model will then be exposed through a server and connected to the Kubernetes scaling workflow to support proactive autoscaling.

Phase 2 will use the completed outputs of Phase 1. The AI cloud operations assistant will consume live Kubernetes metrics, traffic forecasts, scaling decisions, alerts, logs, and resource history. It will support incident management, resource and cost analysis, troubleshooting, natural-language interaction, and controlled operational actions. Any action that changes the cluster will require validation and approval.

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Motivation	1
1.3	Aim	1
2	Background	2
2.1	Introduction	2
2.2	Traffic Prediction	2
2.3	Kubernetes and Autoscaling	2
2.4	Monitoring and Load Testing	2
2.5	AI-Assisted Cloud Operations	2
3	Problem Statement	3
4	Problem Solution	4
4.1	Overview	4
4.2	System Architecture Diagram	5
4.3	Technical Methodology	5
4.4	Phase 1: Predictive Kubernetes Autoscaling	5
4.5	Phase 2: AI Cloud Operations Assistant	8
5	Use Cases	10
5.1	Low Traffic Case	10
5.2	Predicted High Traffic Case	10
5.3	Unexpected Traffic Spike	10
5.4	Service Failure and Incident Management	10
5.5	Approved and Rejected Actions	10
5.6	Evaluation Criteria	11
6	Conclusion	12

Chapter 1

Introduction

1.1 Introduction

This proposal combines Project 9: Kubernetes Autoscaling with Machine Learning Based on Traffic Load Prediction and Project 1: AI-Generated Response and Decision-Making Solution for Cloud Operations into one end-to-end platform. The platform will connect an existing online dataset, traffic forecasting, proactive Kubernetes scaling, and an evidence-based cloud operations assistant.

1.2 Motivation

Changing workloads can affect application response time, reliability, and infrastructure utilization. The project is motivated by the need to prepare capacity before predicted traffic increases and to help engineers interpret operational evidence when diagnosing incidents or deciding how to use resources.

1.3 Aim

1. Prepare and analyze the available online traffic dataset.
2. Build and compare machine learning models for short-term traffic prediction.
3. Deploy a monitored Kubernetes test environment in parallel with model development.
4. Connect the selected model to a prediction server and Kubernetes scaling workflow.
5. Implement proactive autoscaling using predicted traffic and safe scaling limits.
6. Integrate live metrics, forecasts, scaling decisions, alerts, and logs into one assistant backend.
7. Provide incident explanations, troubleshooting support, and resource and cost recommendations.
8. Build a conversational interface for natural-language cloud operations queries.
9. Support defined Kubernetes actions with validation, approval, and audit records.
10. Evaluate the complete platform under normal load, predicted spikes, failures, and excess resource use.

Chapter 2

Background

2.1 Introduction

The project brings together traffic prediction, containerized application deployment, monitoring, autoscaling, and AI-assisted operational analysis. These components form the foundation of the proposed implementation.

2.2 Traffic Prediction

The prediction work will begin with an available online dataset that is independent of Kubernetes. Data preparation, exploratory data analysis, feature engineering, and chronological training, validation, and testing will support the comparison of forecasting models. Persistence and moving-average forecasts will serve as baselines.

2.3 Kubernetes and Autoscaling

A monitored Kubernetes test environment will contain a containerized application and a reactive Horizontal Pod Autoscaler baseline. The project will measure sustainable traffic per pod and pod readiness time. Predicted traffic will then be converted into a bounded replica recommendation through KEDA or a custom-metrics integration, with reactive scaling available as a fallback.

2.4 Monitoring and Load Testing

Prometheus and Grafana will support monitoring and dashboards. Locust scenarios will provide repeatable tests for normal traffic, increasing traffic, and traffic spikes. Metrics, alerts, application logs, events, and scaling history will also supply evidence to the assistant.

2.5 AI-Assisted Cloud Operations

The assistant will interpret forecasts and live operational evidence to support incident explanations, troubleshooting, resource analysis, and estimated-cost recommendations. Defined action requests will be checked against permissions, parameter limits, and current cluster state. Cluster changes will require approval and audit records.

Chapter 3

Problem Statement

Cloud applications experience changing workloads that affect response time, reliability, and infrastructure utilization. Standard Kubernetes autoscaling reacts after metrics such as CPU utilization have already increased. During a rapid traffic increase, users may experience slow responses or errors while additional pods are being created. Keeping excessive resources active at all times also causes waste.

Cloud operations also require engineers to inspect multiple sources, including alerts, logs, metrics, scaling history, and application events. This process is manual and can delay incident diagnosis, troubleshooting, and resource decisions. Operational tools may show what changed without clearly explaining why it happened or what action should be taken.

The proposed project addresses both problems. Machine learning will forecast traffic before the demand arrives, and Kubernetes will use the forecast to prepare application capacity. An AI assistant will then interpret the forecast together with live operational evidence, explain incidents, provide resource and cost recommendations, and propose controlled actions.

Chapter 4

Problem Solution

4.1 Overview

The proposed solution is one integrated cloud operations platform that connects traffic prediction, proactive Kubernetes scaling, operational analysis, and an AI assistant. The system begins with an existing online dataset that is independent of the Kubernetes environment. The dataset will be cleaned and analyzed, and a machine learning model will be trained to forecast future traffic. The selected model will be deployed as a prediction service that returns a timestamped workload forecast through a defined API.

In parallel, the team will build a monitored Kubernetes test environment containing a containerized application, Prometheus, Grafana, Locust, and a reactive Horizontal Pod Autoscaler. When the prediction service becomes available, its forecast will be combined with measured application capacity and scaling limits to calculate the required number of pods. KEDA or a custom-metrics integration will deliver this predictive signal to Kubernetes, allowing the application to prepare capacity before an expected traffic increase. Reactive autoscaling will remain available as a fallback when a prediction is missing or stale.

The same platform will provide an AI cloud operations assistant. The assistant will use live Kubernetes metrics, traffic forecasts, scaling recommendations, actual scaling actions, alerts, logs, and application events as one evidence set. It will explain current conditions, summarize incidents, suggest likely causes, analyze CPU, memory, pod usage, and estimated cost, and answer operational questions through a conversational interface.

When the assistant recommends an operational action, the recommendation will be converted into a defined request, such as changing a replica count or creating an incident record. A validation layer will check permissions, parameter limits, and current cluster state. Actions that change the cluster will require approval and will be written to an audit log. This creates one complete flow from historical data and workload prediction to scaling, explanation, decision support, and controlled execution.

4.2 System Architecture Diagram

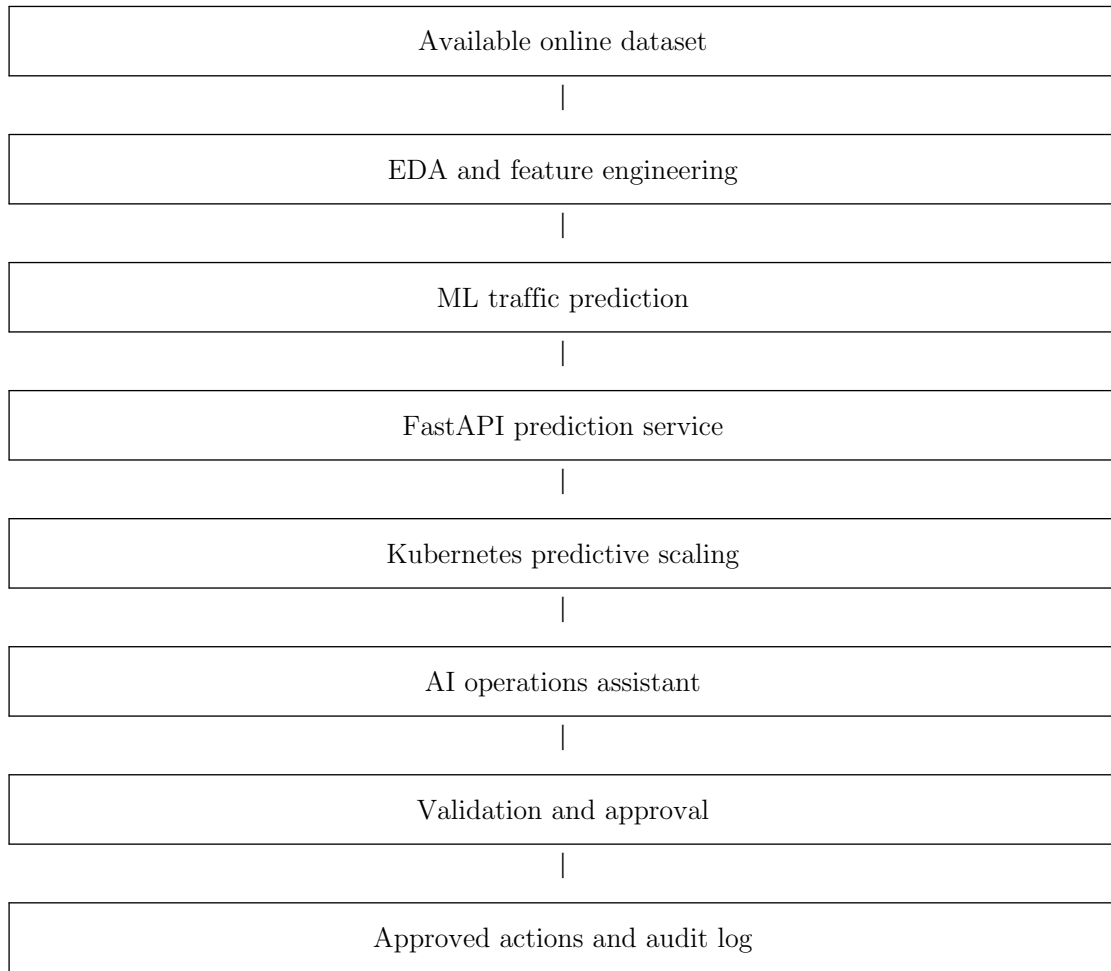


Figure 4.1: Proposed end-to-end system architecture.

4.3 Technical Methodology

The technical methodology is organized into the two implementation phases and their approved milestones.

Phase 1 builds the prediction and Kubernetes scaling foundation. Phase 2 uses those outputs to implement the cloud operations assistant and controlled decision-making functions.

4.4 Phase 1: Predictive Kubernetes Autoscaling

Phase 1 begins with the available online dataset; the dataset will not be extracted from Kubernetes.

Dataset analysis and model development will proceed alongside Kubernetes environment development so that integration can begin immediately after the prediction API is ready.

Milestone 1 - Data Preparation and EDA

This milestone prepares the available online dataset for time-series prediction. It includes:

- Inspecting the columns, timestamps, sampling interval, target variable, and data range.
- Removing duplicated records and correcting invalid values.
- Handling missing data and aligning records to a consistent time interval.
- Transforming data types and creating a clean analysis dataset.
- Performing EDA for traffic trends, peaks, seasonality, distributions, correlations, and anomalies.
- Documenting dataset quality, assumptions, and limitations.

Milestone result: A documented, model-ready dataset and an EDA report with clear visualizations.

Milestone 2 - Machine Learning Model Development

This milestone develops and selects the traffic prediction model. It includes:

- Creating lag features, rolling statistics, recent-change features, and suitable time features.
- Dividing the data chronologically into training, validation, and test sets.
- Implementing persistence and moving-average forecasts as baselines.
- Training candidate models such as linear regression, Random Forest, Gradient Boosting, and XGBoost.
- Comparing models using MAE, RMSE, peak-period error, underprediction frequency, and inference time.
- Selecting, saving, and documenting the best model.

Milestone result: A validated traffic prediction model with documented metrics and comparison against baselines.

Milestone 3A - Prediction Server

This milestone makes the selected model available to the rest of the platform. It includes:

- Packaging the trained model and preprocessing steps.
- Building a FastAPI prediction service.
- Defining request validation and error handling.
- Returning the prediction timestamp, forecast horizon, predicted traffic, model version, and data-freshness status.
- Testing the service with valid, missing, and stale inputs.

Milestone result: A working prediction API that returns valid structured forecasts.

Milestone 3B - Kubernetes Environment in Parallel

This milestone runs in parallel with Milestones 1, 2, and 3A. It includes:

- Containerizing and deploying a test web application to Kubernetes.
- Configuring resource requests, limits, health checks, and readiness checks.
- Installing Prometheus and Grafana for monitoring.
- Creating Locust scenarios for normal traffic, increasing traffic, and traffic spikes.
- Configuring a reactive HPA baseline.
- Measuring sustainable traffic per pod and pod readiness time.

Milestone result: A monitored Kubernetes environment that supports repeatable load tests and reactive scaling.

Milestone 4 - Model, Server, and Kubernetes Integration

This milestone connects the prediction output to the scaling workflow. It includes:

- Calling the prediction API on a defined schedule.
- Rejecting unavailable or stale forecasts.
- Converting predicted traffic into a bounded replica recommendation.
- Publishing the scaling signal through KEDA or Kubernetes custom metrics.
- Applying minimum and maximum replicas, safety margins, stabilization, and reactive fallback.
- Recording forecasts, recommendations, scaling actions, and outcomes.

The initial replica recommendation will be calculated as follows:

$$N_{\text{recommended}} = \min(N_{\text{max}}, \max(N_{\text{min}}, \text{ceil}(\text{predicted traffic} \times s / C_{\text{pod}})))$$

where predicted traffic is the forecast for time $t + h$, s is a safety factor, C_{pod} is measured pod capacity, and N_{min} and N_{max} are the replica limits. Stale or unavailable forecasts will be rejected and reactive scaling will provide fallback behavior.

Milestone result: An end-to-end predictive autoscaling pipeline with safety limits, logging, stale-prediction handling, and reactive fallback.

Phase 1 Deliverables

1. Cleaned dataset and EDA report.
2. Reproducible preprocessing, feature-engineering, and model-comparison pipeline.
3. Selected traffic prediction model and documented FastAPI service.
4. Containerized test application and monitored Kubernetes environment.
5. Grafana dashboards, Locust scenarios, reactive HPA baseline, and pod-capacity measurements.
6. Predictive autoscaling integration with safety limits, fallback behavior, and Phase 1 evaluation.

4.5 Phase 2: AI Cloud Operations Assistant

Phase 2 consumes the outputs of Phase 1. The assistant will use the same prediction service, Kubernetes metrics, and scaling records rather than rebuilding these components.

Milestone 1 - Integrate Project 9

- Connect the assistant backend to Kubernetes metrics, traffic forecasts, scaling recommendations, and scaling history.
- Create structured functions for current cluster status and predicted workload.
- Validate timestamps, freshness, and unavailable-data behavior.

Milestone result: The assistant can read live status and predicted workload.

Milestone 2 - Incident Management

- Collect relevant alerts, application logs, Kubernetes events, and metrics.
- Group related evidence into one incident context.
- Summarize what happened and suggest likely causes supported by the evidence.
- Provide troubleshooting steps and identify missing information.

Milestone result: A user can ask “Why is the service slow?” and receive an evidence-based answer.

Milestone 3 - Resource and Cost Analysis

- Analyze CPU, memory, pod usage, replica history, and predicted demand.
- Identify unused capacity, repeated overprovisioning, and expected shortages.
- Estimate relative infrastructure cost from resource and scaling history.
- Produce practical resource and cost recommendations.

Milestone result: The assistant produces resource and cost recommendations.

Milestone 4 - Conversational Assistant

- Build the chat interface and conversation backend.
- Connect user questions to monitoring, forecasting, incident, and resource-analysis functions.
- Return concise answers that identify the metrics, forecasts, logs, or events used as evidence.

Milestone result: A working chatbot for monitoring, incidents, and troubleshooting.

Milestone 5 - Decision Support and Actions

- Convert recommendations into defined action types, such as changing replica counts or creating an incident record.
- Check permissions, parameters, safety limits, and current cluster state.
- Require approval before applying changes to the cluster.

- Execute approved actions and record the request, approval, result, and evidence in an audit log.

Milestone result: The assistant can propose an action, explain why, and execute it after approval.

Milestone 6 - Evaluation and Demonstration

- Test normal traffic, predicted spikes, unexpected spikes, service failures, and excess resource usage.
- Compare assistant answers with recorded metrics, alerts, logs, forecasts, and outcomes.
- Verify correct approval and execution behavior for allowed and rejected actions.
- Prepare the final end-to-end demonstration and results.

Milestone result: Evaluation results and an end-to-end demonstration.

Phase 2 Deliverables

1. Integrated assistant backend with access to Phase 1 evidence.
2. Incident identification, summarization, and likely-cause analysis module.
3. Resource-utilization, estimated-cost, and recommendation module.
4. Evidence-based conversational monitoring and troubleshooting interface.
5. Defined actions with approval, permissions, safety limits, execution, and audit logging.
6. Scenario-test results, end-to-end demonstration, and final technical documentation.

Chapter 5

Use Cases

5.1 Low Traffic Case

The system will be tested under low traffic to assess unused capacity, resource utilization, replica-hours, and scaling stability. The assistant will identify potential overprovisioning and provide resource and estimated-cost recommendations.

5.2 Predicted High Traffic Case

A repeatable traffic increase will test whether the forecast allows additional pods to become ready before demand arrives. The demonstration will trace the forecast, bounded replica recommendation, scaling action, assistant explanation, and observed application performance.

5.3 Unexpected Traffic Spike

An unexpected spike will test the reactive autoscaling baseline and fallback behavior. The evaluation will record p95 and p99 response time, error rate, readiness time, and time above the latency target.

5.4 Service Failure and Incident Management

A service failure will test the assistant's ability to combine alerts, logs, metrics, and Kubernetes events into an incident explanation. Its suggested causes and troubleshooting steps will be checked against recorded evidence.

5.5 Approved and Rejected Actions

Action scenarios will test parameter validation, permissions, replica limits, current-state checks, and approval enforcement. Approved actions and rejected requests will be checked against the audit log.

5.6 Evaluation Criteria

Criterion	Measurement
Traffic prediction quality	MAE, RMSE, peak-period error, and underprediction frequency.
Application performance	p95 and p99 response time, error rate, and time above the latency target.
Scaling performance	Replica readiness time, scaling frequency, stability, and behavior during missing predictions.
Resource efficiency	Replica-hours, CPU and memory utilization, and unused capacity.
Incident responses	Agreement between the assistant’s explanation and recorded alerts, logs, metrics, and events.
Recommendations	Relevance of the proposed resource, cost, and troubleshooting recommendations.
Action safety	Correct enforcement of approval, permissions, parameter validation, replica limits, and audit logging.
End-to-end integration	Successful flow from forecast to scaling decision, assistant explanation, approved action, and observed outcome.

Chapter 6

Conclusion

The final system will demonstrate how an existing online dataset can be used to train a traffic prediction model, how that model can drive proactive Kubernetes scaling, and how an AI cloud operations assistant can use forecasts and live operational evidence to support monitoring, incident management, troubleshooting, resource optimization, and controlled actions.

The final demonstration will show the complete flow: the model predicts a workload increase, the Kubernetes decision layer recommends additional replicas, the assistant explains the forecast and the reason for the decision, an approved action is applied, and the dashboard displays the resulting effect on performance and resource utilization.